

```

"""
visualize.py
-----
シミュレーション結果の可視化。
results/simulation_log.json を読んでグラフを生成する。
"""

import json
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from pathlib import Path

def plot_results(log_path: str = "results/simulation_log.json", save_path: str =
    with open(log_path, encoding="utf-8") as f:
        log = json.load(f)

    gens          = [d["generation"]          for d in log]
    mean_H         = [d["mean_H"]              for d in log]
    mean_Z         = [d["mean_Z"]              for d in log]
    z_ratio        = [d["Z_ratio"]             for d in log]
    env_factor     = [d["env_factor"]          for d in log]
    anomaly_mean   = [d["mean_anomaly_score"]  for d in log]
    anomaly_max    = [d["max_anomaly_score"]   for d in log]
    stability      = [d["mean_stability"]      for d in log]

    # アノマリーテキストのタイムライン（上位テキスト）
    top_texts = [(d["generation"], d["top_anomaly_text"]) for d in log]

    fig = plt.figure(figsize=(16, 12), facecolor="#0a0a18")
    fig.suptitle(
        "Anomaly Soul Kit - LLM Anomaly Evolution Report",
        fontsize=14, color="white", fontweight="bold", y=0.98
    )
    gs = gridspec.GridSpec(3, 3, hspace=0.55, wspace=0.38, figure=fig)

    def style(ax, title):
        ax.set_facecolor("#10102a")
        ax.tick_params(colors="#9999cc", labelsize=8)
        ax.spines[:].set_color("#2a2a4a")
        ax.set_title(title, color="#bbbbff", fontsize=9, pad=7)
        ax.grid(color="#1a1a35", linestyle="--", linewidth=0.6)
        ax.set_xlabel("Generation", color="#8888aa", fontsize=8)

```

```

# — 1. mean_H
ax = fig.add_subplot(gs[0, 0])
style(ax, "mean_H (Vitality)")
ax.plot(gens, mean_H, color="#4fc3f7", lw=1.8)
ax.fill_between(gens, mean_H, alpha=0.12, color="#4fc3f7")
ax.set_ylabel("H", color="#8888aa", fontsize=8)

# — 2. mean_Z
ax = fig.add_subplot(gs[0, 1])
style(ax, "mean_Z (Anomaly Intensity)")
ax.plot(gens, mean_Z, color="#ce93d8", lw=1.8)
ax.fill_between(gens, mean_Z, alpha=0.12, color="#ce93d8")
ax.set_ylabel("Z", color="#8888aa", fontsize=8)

# — 3. Z_ratio
ax = fig.add_subplot(gs[0, 2])
style(ax, "Z Awakening Ratio")
ax.plot(gens, z_ratio, color="#80cbc4", lw=1.8)
ax.fill_between(gens, z_ratio, alpha=0.12, color="#80cbc4")
ax.axhline(0.5, color="#ffffff22", ls=":", lw=1)
ax.set_ylim(0, 1.05)
ax.set_ylabel("ratio", color="#8888aa", fontsize=8)

# — 4. アノマリースコア推移
ax = fig.add_subplot(gs[1, 0])
style(ax, "Anomaly Score (mean / max)")
ax.plot(gens, anomaly_mean, color="#ef9a9a", lw=1.5, label="mean")
ax.plot(gens, anomaly_max, color="#ff5252", lw=1.2, ls="--", label="max", al
ax.set_ylim(0, 1.05)
ax.set_ylabel("score [0-1]", color="#8888aa", fontsize=8)
ax.legend(facecolor="#1a1a3a", labelcolor="white", fontsize=7)

# — 5. 環境変動 overlay
ax = fig.add_subplot(gs[1, 1])
style(ax, "Environment Factor")
ax.plot(gens, env_factor, color="#a5d6a7", lw=1.0, alpha=0.85)
ax.fill_between(gens, env_factor, alpha=0.1, color="#a5d6a7")
ax.set_ylabel("env", color="#8888aa", fontsize=8)

# — 6. stability 進化
ax = fig.add_subplot(gs[1, 2])
style(ax, "Stability Evolution")
ax.plot(gens, stability, color="#ffcc80", lw=1.8)
ax.fill_between(gens, stability, alpha=0.12, color="#ffcc80")
ax.axhline(1.0, color="#ffffff22", ls=":", lw=1)
ax.set_ylabel("stability", color="#8888aa", fontsize=8)

```

```

# — 7. アノマリーテキストタイムライン (散文)
ax_text = fig.add_subplot(gs[2, :])
ax_text.set_facecolor("#08081a")
ax_text.axis("off")
ax_text.set_title(
    "Top Anomaly Expressions Timeline (highest anomaly score per generation)
    color="#bbbbff", fontsize=9, pad=8
)

# サンプリング: 等間隔で最大8世代
step = max(1, len(top_texts) // 8)
samples = top_texts[::step][:8]

y_pos = 0.92
for gen, text in samples:
    display = text[:110] + ("..." if len(text) > 110 else "")
    ax_text.text(
        0.01, y_pos,
        f"Gen {gen:3d} | {display}",
        color="#ccccee", fontsize=7.5,
        transform=ax_text.transAxes,
        verticalalignment="top",
        fontfamily="monospace",
    )
    y_pos -= 0.115

plt.savefig(save_path, dpi=150, bbox_inches="tight", facecolor="#0a0a18")
print(f"グラフ保存: {save_path}")
return save_path

if __name__ == "__main__":
    plot_results()

```